

Sistemas Operativos: Práctica 2

Shaofan Xu

Javier Santamaria

April 9, 2026

1 Introducción

El objetivo de esta práctica es utilizar mecanismos de sincronización para simular una block chain. En este entorno diversos procesos se forman mineros para competir por el premio, así utilizando señales y semáforos para comunicar entre los procesos.

2 Requisitos cumplidos y no cumplidos

2.1 Requisitos cumplidos

Se han implementado correctamente los siguientes objetivos principales de la práctica:

- **Gestión de múltiples procesos Minero:** Se ha implementado un fichero compartida protegido por mutex donde los procesos registran pid al entrar al sistema y eliminan al salir del sistema cuando llegan su tiempo límite, así destruyendo el fichero el último proceso que sale.
- **Proceso de minería:** el primer proceso establece un objetivo inicial (0 en nuestro caso), y lanza señal para iniciar la ronda, posteriormente el primer minero que se encuentra el resultado escribe su resultado en target.tgt y lanzando otra señal para empezar la votación.
- **Sistema de votación:** una vez empezado el proceso de votación los procesos participantes deben comprobar el resultado del minero candidato para votar si esta a favor o en contra, en cuyo caso, el proceso candidato irá comprobando los votos teniendo un máximo número de intento hasta tener más de un 50% de voto a favor. Una vez tenido el resultado, el proceso candidato convierte en ganador, guardando su resultado en un fichero cuyo nombre es su pid, y el target de la siguiente ronda en target.tgt
- **Protección de zona crítica:** Para que el sistema no falle por problemas de concurrencia ni pierda señales por el camino, hemos protegido las zonas críticas del código de cuatro formas:
 - **Bloqueo inicial de señales:** Antes de entrar al bucle principal, se crea una máscara con SIGUSR1, SIGUSR2 y SIGALRM y usando sigprocmask. Así nos aseguramos de que, si llega una señal justo cuando esta leyendo o escribiendo en un fichero compartido, esta se quede pendiente y no rompe la ejecución a medias.
 - **Control de interrupciones (EINTR):** Al mezclar semáforos y señales, pasaba que a veces la llamada a sem_wait() se interrumpía por la llegada de una señal antes de haber conseguido el semáforo. Para evitar que el código siga adelante asumiendo que

tiene el recurso (lo que corrompería todo), protegemos las peticiones en un bucle: `while (sem_wait(...) == -1 && errno == EINTR);`. Así, si una señal despierta al proceso por error, este vuelve a intentar.

- **Espera pasiva con `sigsuspend`:** Para cumplir con el enunciado y no freír la CPU con esperas activas, cuando el minero no está minando, ni votando, ni gestionando la victoria, se pone a dormir llamando a `sigsuspend(&mask_empty)`. Esto vacía la máscara de bloqueos temporalmente y deja al proceso suspendido hasta que llega una señal que lo despierta para la siguiente fase.
- **Evitar deadlocks en los hilos:** Durante las pruebas a menudo sucede que un proceso estaba bloqueado en el `pthread_join` esperando a sus hilos y otro minero ganaba la ronda, el proceso bloqueado no podía recibir la señal `SIGUSR2` para ir a votar (se quedaba colgado). Como solución propuesta se desbloquea las señales con `SIG_UNBLOCK` justo antes del join y volviéndolas a bloquear después. De esta forma, la orden de votación entra sin problemas, los hilos abortan su ejecución y el programa no se atasca.

2.2 Requisitos no cumplidos

Todos los requisitos obligatorios especificados en el enunciado han sido implementados y funcionan correctamente.

3 Pruebas: Funcionamiento y Rendimiento

3.1 Entorno de ejecución

Las pruebas y mediciones se han llevado a cabo bajo las siguientes condiciones de hardware y software para garantizar la consistencia de los resultados:

- Sistema Operativo: WSL2 (Ubuntu) sobre Windows 11.
- Hardware: 16 GB de memoria RAM.
- Herramienta de medición: Comando `time` de la shell de Linux.

3.2 Pruebas de Funcionamiento

Se han ejecutado diversas pruebas para asegurar el correcto funcionamiento del programa.

3.2.1 Casos normales

Se ejecutó el programa con parámetros correspondientes:

```
./miner 3 8 & ./miner 5 8 & ./miner 10 8 & ./miner 10 1
```

El programa generó correctamente los archivo .log con la estructura especificada(Id, ronda, target, solución, etc), y la validación (accepted/rejected). Así imprimiendo en la terminal exactamente los mensajes requeridos:

```

● → practica2 git:(main) ✖ make test
./miner 3 8 & ./miner 5 8 & ./miner 10 8 & ./miner 10 1
Miner 48017 added to system
Miner 48014 added to system
Miner 48016 added to system
Miner 48015 added to system
Winner 48016 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48016 -> [ Y Y Y ] -> Accepted
Winner 48016 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48015 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48016 -> [ Y Y Y ] -> Accepted
Winner 48016 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Winner 48014 -> [ Y Y Y ] -> Accepted
Miner 48014 exited system. Remaining miners: 3
Miner 48015 exited system. Remaining miners: 2
Miner 48017 exited system. Remaining miners: 1
Miner 48016 was the last one. System cleaned.

```

Figura 1:

Ejecución de programa demostrando el proceso de entrada de los procesos mineros, los procesos minando, y destrucción del fichero cuando termina el último proceso

3.2.2 Casos de errores

Se probaron situaciones anómalas como:

- **Falta de argumentos:** Al ejecutar `./miner` sin parámetros, el programa detecta el error, muestra el uso correcto `"Usage ./miner < N_SECS > < N_THREADS >".`

3.3 Pruebas adicionales

[illegible]

Figura 2:

Ejecución de 4 mineros simultáneos durante 10 segundos. Se lanzaron dos procesos con 8 hilos y dos procesos con 1 hilo. Como se observa en las líneas de 'Winner', los PIDs correspondientes a los procesos con 8 hilos acaparan la inmensa mayoría de las victorias en las rondas, demostrando que un mayor número de hilos permite explorar el espacio de hashes de forma mucho más rápida y eficiente, ganando la condición de carrera por el semáforo sem winner.

```
dsadas@Santa:/mnt/c/Users/jsant/OneDrive/Documentos/Prog/Tercero/SOPER/I2/SO/practica2$ tail -n 15 8129.txt
Id:      39
Winner:  8129
Target:  3419216
Solution: 3308492 (validated)
Votes:   3/3
Wallets: 8129:39

Id:      40
Winner:  8129
Target:  4505593
Solution: 8924837 (validated)
Votes:   3/3
Wallets: 8129:40

dsadas@Santa:/mnt/c/Users/jsant/OneDrive/Documentos/Prog/Tercero/SOPER/I2/SO/practica2$
```

Figura3:

Registro de victorias generado por el proceso 8129. Como se puede observar, el minero ha registrado correctamente el 'Target' y la 'Solution' de cada ronda, contabilizando los votos a favor (Y) de los demás procesos. El campo 'Wallets' demuestra que el incremento de monedas locales se gestiona de forma segura tras la validación de la red.

```
dsadas@Santa: /mnt/c/Users/jsant/OneDrive/Documentos/Prog/Tercero/SOPER/I2/S0/practica2$ ./miner 8 4 & sleep 3 ; ./miner 2 4
[1] 12348
Miner 12348 added to system
Miner 12361 added to system
Winner 12361 -> [ Y ] -> Accepted
Winner 12348 -> [ Y ] -> Accepted
Winner 12361 -> [ Y ] -> Accepted
Winner 12348 -> [ Y ] -> Accepted
Winner 12361 -> [ Y ] -> Accepted
Winner 12348 -> [ Y ] -> Accepted
Winner 12361 -> [ Y ] -> Accepted
Winner 12348 -> [ Y ] -> Accepted
Winner 12361 -> [ Y ] -> Accepted
Winner 12348 -> [ Y ] -> Accepted
Winner 12361 -> [ Y ] -> Accepted
Winner 12348 -> [ Y ] -> Accepted
Winner 12361 -> [ Y ] -> Accepted
Winner 12348 -> [ Y ] -> Accepted
Winner 12361 -> [ Y ] -> Accepted
Miner 12361 exited system. Remaining miners: 1
dsadas@Santa: /mnt/c/Users/jsant/OneDrive/Documentos/Prog/Tercero/SOPER/I2/S0/practica2$ Winner 12348 -> [ ] -> Accepted
Winner 12348 -> [ ] -> Accepted
Winner 12348 -> [ ] -> Accepted
Winner 12348 -> [ ] -> Accepted
Winner 12348 -> [ ] -> Accepted
Miner 12348 was the last one. System cleaned.
^C
[1]+  Done                  ./miner 8 4
dsadas@Santa: /mnt/c/Users/jsant/OneDrive/Documentos/Prog/Tercero/SOPER/I2/S0/practica2$
```

Figura4:

Esta ejecución confirma que el sistema es robusto y no sufre bloqueos o *deadlocks*. Los procesos se registran y votan de forma concurrente sin problemas , y si alguno agota su tiempo, sale ordenadamente permitiendo que los demás sigan trabajando. Si un minero se queda solo, detecta que no hay votos pendientes y auto-aprueba las rondas para no quedarse colgado esperando señales que nunca llegarán. Finalmente, el último proceso en salir se encarga de realizar la limpieza de semáforos y ficheros temporales, evitando fugas de memoria y dejando el entorno totalmente limpio.

4 Conclusión y Respuesta a las cuestiones del enunciado

Pregunta a)

¿Se produce algún problema de concurrencia en el sistema descrito?

Sí, el sistema tiene varios puntos donde la concurrencia puede causar fallos. El problema principal es el acceso a los ficheros compartidos como `pids.pid`, `target.tgt` y `vote.txt`, ya que al ser procesos independientes, varios podrían intentar escribir o leer a la vez, corrompiendo los datos. También hay un problema de "condición de carrera" al final de cada ronda: varios mineros pueden encontrar una solución válida casi al mismo tiempo y ambos intentarían proclamarse ganadores simultáneamente.

Pregunta b)

¿Cómo se ha solucionado en el planteamiento dado?

La solución se basa en el uso de semáforos con nombre de POSIX que actúan como mutex para garantizar la exclusión mutua. Cada vez que un proceso necesita acceder a un fichero, debe adquirir el semáforo correspondiente (sem_pid, sem_target, sem_votes). Para la competición por la victoria, se utiliza el semáforo sem_winner con la función sem_trywait, lo que asegura que solo el primer minero en realizar la llamada se convierta en el ganador oficial. Asimismo, se implementó un bloqueo de señales y el uso de sigsuspend para evitar la pérdida de notificaciones críticas durante la ejecución de estas secciones.

Pregunta c)

¿Es sencillo, o siquiera factible, organizar un sistema de este tipo usando únicamente señales?

No es nada sencillo y, en la práctica, no es factible para un sistema así. Las señales son solo avisos; no pueden enviar datos como la solución del hash o los votos de cada minero. Además, las señales no sirven para bloquear el acceso a un archivo; si usáramos solo señales, no tendríamos forma de garantizar que dos procesos no escriban en el mismo fichero a la vez, lo que acabaría rompiendo toda la lógica de la cadena.

Pregunta d)

¿Cuáles son los mineros que tienden a ganar más monedas? (mostrar casos de ejecución que corroboren la respuesta)?

Como se puede ver en las pruebas de rendimiento, los mineros que más ganan son los que tienen más hilos (N_THREADS). Al tener más hilos, el proceso divide el trabajo de búsqueda y puede probar muchos más hashes por segundo en paralelo. En mi captura de la Figura 2, se ve claramente que los procesos 8128 y 8129 (que lanzamos con 8 hilos) ganan casi todas las rondas, mientras que los de 1 hilo (8130 y 8131) casi nunca llegan a tiempo de encontrar la solución